Ders 9 Genişletilmiş Notu: İşlemci Tasarımına Giriş: Datapath, Aritmetik Devre Tasarımı, ALU, Kaydırıcı ve Kontrol Kelimesi

Ders 9 Genişletilmiş Notu: İşlemci Tasarımına Giriş: Datapath, Aritmetik Devre Tasarımı, ALU, Kaydırıcı ve Kontrol Kelimesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. İşlemci Tasarımına Geçiş
- 2. CPU'nun İki Bloğu ve Dış Yolları
- 3. Örnek Datapath: Dört Register'lı Yapı
- 4. Aritmetik Devre Tasarımı
- 5. Alıştırma 1: Hem A Hem B Giriş Lojiği Olan Devre
- 6. Alıştırma 2: Dört Bitlik Aritmetik Devre
- 7. Alıştırma 3: Fonksiyondan İşlemi Bulma
- 8. Giriş Lojiğindeki C_{in} Nereden Gelir?
- 9. Lojik Blok Tasarımı
- 10. Aritmetik ve Lojik Bloğu Birleştirmek: ALU
- 11. Kombinasyonel Kaydırıcı (Shift Unit)
- 12. Barrel Shifter
- 13. Genel Datapath ve 16 Bitlik Kontrol Kelimesi
- 14. Kontrol Kelimesinin Kodlama Tablosu
- 15. Mikro İşlem Örnekleri
- 16. İkili Kodlu Gösterim ve Alıştırma 4

1. İşlemci Tasarımına Geçiş

Hoca, şimdiye kadar anlatılan her şeyin aslında bu konuya hazırlık olduğunu söyleyerek başladı. Bilgisayarın işlemci tasarımı kısmına geçiyoruz. Bu hafta ve sonraki haftalarda işlemci tasarımını ve işlemciyi hızlandırmak için kullanılan teknolojileri göreceğiz: boru hattı işleme (pipeline processing), önbellek (cache memory) ve sanal bellek (virtual memory) gibi.

> **Hocanın vurgusu:** Bu konular mesleğinizin temelini oluşturur. Yalnız ders notlarıyla çalışmaya kalkarsanız çok yorulur, çok vakit kaybedersiniz. Dersi derste öğrenmeye çalışın.

Mano'nun kitabı bu kavramları belli bir işlemciye bağlı kalmadan, genel olarak ve birbirinden farklı yapılar üzerinden anlatır. Hocanın deyişiyle bunlar "beynelminel" kavramlardır. Bunları öğrenen biri A işlemcisinin nasıl çalıştığını, B işlemcisinin ondan nasıl farklı çalıştığını çok hızlı kavrayabilir. Dersin izlediği bölüm kitabın 9. bölümüdür.

2. CPU'nun İki Bloğu ve Dış Yolları

Bir merkezi işlem biriminin (CPU) içinde iki temel blok vardır: veri yolu birimi (datapath unit) ve kontrol birimi (control unit). Hoca bunlardan en önemlisinin, bu hafta üzerinde çalışılacak datapath birimi olduğunu söyledi. İki blok önceki haftalarda kavramsal olarak görülmüştü, artık tasarımlarına başlanıyor.

İki blok arasındaki ilişki şöyledir: Kontrol birimi datapath birimine komut gönderir. Datapath birimi gelen komutları işleyerek bir sonuç üretir ve durum bilgisini kontrol birimine geri gönderir. Temel yapı bu iletişim işaretleriyle çalışır.

CPU'yu dışarıdan bakınca bir entegre olarak çizersek üç yol görürüz:

- **Veri yolu (data bus):** Çift yönlüdür. Genişliği çok önemlidir, çünkü işlemcinin kaç bitlik olduğunu gösterir.
- **Adres yolu (address bus):** CPU'dan dışarıya doğru yönlendirilmiştir. Genişliği bakımından o da temel önemdedir.
- **Kontrol yolu (control bus):** İşlemcinin çevre birimleriyle iletişimde kullandığı kontrol işaretlerini taşır: Memory Read, Memory Write, I/O Read, I/O Write ve doğrudan bellek erişimi (Direct Memory Access) için kullanılan işaretler. Bunlar ilerleyen haftalarda ayrıntılı görülecek.

Bu haftanın konusu datapath birimini tasarlamaktır. Bu birimin içinde kabaca şunlar bulunur:

- **Register kümesi (register file):** Hoca adın yanıltmaması için uyardı: Burada "file" bir dosya değil, register kümesi anlamındadır. Register'ların sayısı ve adları işlemciden işlemciye değişir.
- **Aritmetik lojik birim (ALU).**
- **Kaydırma birimi (shift unit):** Kaydırma (shift) ve döndürme (rotate) işlemlerini yapan bloktur.

3. Örnek Datapath: Dört Register'lı Yapı

Kitabın 9. bölümü doğrudan oldukça kapsamlı bir örnek datapath şekliyle başlar. Hoca ilk bakışta karışık görünebileceğini, ama on beş dakika sonra "ne basit bir devreymiş" diyeceğimizi söyledi.

3.1 Register Kümesi ve Kaynak Seçimi

Örnekte dört register vardır: R0, R1, R2, R3. Bu register'ların içeriğini (content) aritmetik işlem bloğuna gönderip işlem yapmak istiyoruz. Bunun için önce hangi register'ların işleme alınacağını seçebilmemiz gerekir.

Seçme işini iki çoklayıcı (multiplexer) yapar. Bir işleme iki işlenen (operand) katılacağı için iki MUX vardır. Dört register arasından seçim yapacakları için bunlar 4'ten 1'e MUX'tur ve her birinin 2 bitlik seçme ucu vardır. Bu uçların adı şekilde A select ve B select'tir. Seçilen değerler A data ve B data olarak adlandırılan yollara çıkar. Birinin çıkışı doğrudan aritmetik devreye gider, ötekinin çıkışı ise önce ikinci bir MUX'a girer.

Register sayısı devrenin ihtiyacına göre verilmiş bir karardır. Dört yerine sekiz register olsaydı 8'den 1'e MUX kullanılırdı.

3.2 MUX B ve Sabit Sayı Girişi

Her işlemi register'lar arasında yapmak zorunda değiliz. İşleme dışarıdan sabit bir sayı da girebilir. Hocanın örneği: R3 register'ına sabit 3 sayısını eklemek. Bu sabit, sabit sayı girişi (constant in) üzerinden verilir.

> **Ders sonrası not:** Hoca bu örnekte sonucun R4'e yazılabileceğini söyledi (00:22:08). Bu datapath'te yalnız R0-R3 vardır, R4 yoktur. Hoca aynı işlemi 00:31:06'da hedefi R0 alarak tekrar kullandı.

Register ile sabit arasındaki seçimi MUX B yapar. İki kaynak olduğu için 2'den 1'e MUX'tur ve tek bitlik bir seçme ucu vardır (MB select). Seçtiği iki değer de n bitliktir. Böylece aşağıdaki aritmetik bloğa A yolundan A data, B yolundan MUX B'nin çıkışı gelir.

3.3 Fonksiyon Birimi: ALU

Şekilde mavi renkli fonksiyon birimi (function unit) iki bloktan oluşur. Birincisi aritmetik lojik birimdir. ALU'nun girişlerine n bitlik A ve B sayıları gelir. 4 bitlik G select ucuyla içeride 16 farklı işlem seçilebilir. Bunlar aritmetik ya da lojik işlemler olabilir ve sonuç n bitlik çıkışta oluşur. ALU'nun iç yapısı bu dersin devamında tasarlanacak.

> **Ders sonrası not:** Hoca bu bloğu tanıtırken "toplama, çıkarma, çarpma, bölme neyse" dedi. Bu derste tasarlanan ALU'nun 14. bölümdeki işlem tablosunda çarpma ve bölme yoktur.

3.4 Fonksiyon Birimi: Shifter

Aritmetik işlemler bir tarafta, kaydırma işlemleri öbür tarafta yapılır. Shifter bloğuna yalnız B yolundan n bitlik bir giriş gelir, çünkü kaydırma için tek işlenen yeterlidir: Seçilen bir register sağa ya da sola kaydırılır veya döndürülür. Shifter'ın 2 bitlik H select ucu vardır, bu uçla dört farklı kaydırma işlemi seçilebilir.

Kaydırmada, kaydırma yönüne göre boşalan yere seri giriş (serial input) olarak 0 girer.

> **Ders sonrası not:** Hocanın sözü altyazıda "sola shift yapıyorsanız soldan bir sıfır girişi" diye geçiyor (00:26:40). Sola kaydırmada boşalan bit en sağdaki bittir, yani 0 sağdan girer. Sağa kaydırmada boşalan bit en soldakidir, 0 soldan girer.

3.5 MUX F

Shifter'ın H çıkışı ile ALU'nun çıkışı 2'den 1'e bir MUX'a, F multiplexer'ına girer. Bu MUX sonucun ALU'dan mı shifter'dan mı alınacağını seçer. İki kaynak olduğu için tek bitlik seçme ucu (MF select) yeterlidir. Seçtiği iki sayı da n bitliktir ve çıkışı da n bitliktir.

Soru: Veriyi fonksiyon birimine sokarken bir MUX kullanmadık. O zaman gönderdiğimiz veri her durumda hem shifter'dan hem aritmetik lojik bloktan geçiyor, sonra MUX hangisinin alınacağına mı karar veriyor?

Cevap: Doğru. İki blok birbirine paraleldir ve aynı anda sonuç üretir. Biz bunlardan birini MUX F ile seçeriz.

3.6 MUX D ve Dışarıdan Gelen Veri

Son katmanda bir MUX daha vardır: D multiplexer'ı. Bu da 2'den 1'e bir MUX'tur ve iki n bitlik değer arasında seçer: fonksiyon biriminin çıkışı ya da data in kanalıyla gelen veri. Data in'in kaynağı bellek ya da bir G/Ç birimi (I/O device) olabilir. Bellekten veya G/Ç biriminden gelen bir veriyi iç register'a aktarmak için bu yol kullanılır.

3.7 Hedef Register'ın Seçimi

MUX D'nin çıkışı yukarıya döner ve bütün register'ların data in girişine bağlanır. Yani sonuç her register'ın girişinde hazır bekler. Hangisine yükleme yapılacağına load enable ucu ve hedef seçme çözücüsü (destination select decoder) birlikte karar verir.

Çözücünün 2 bitlik girişi vardır. Hocanın örneği: Destination select 00 ise çözücünün 0 numaralı çıkışı aktif (1), ötekiler 0 olur. Çözücü çıkışları ile load enable'ı alan AND kapılarının çıkışı her register'ın yükleme işaretini oluşturur. Bu durumda yalnız en üstteki AND kapısının çıkışı 1 olur ve yalnız R0'a yükleme yapılır. Öteki AND kapılarının girişi 0 olduğu için diğer register'lara yükleme yapılmaz.

Hocanın bütün şekli bağlayan örneği: R0'a R3 + 3 yazmak istiyorsak kaynak register olarak R3'ü, MUX B'de sabit sayı girişini, hedef register olarak R0'ı seçeriz. Fonksiyon biriminde de işlemin toplama olarak seçilmesi gerekir.

3.8 Durum Bitleri ve Sıfır Algılama

Yapılan aritmetik ya da lojik işlemin sonucuna göre dört durum biti güncellenir: taşma (overflow, V), elde (carry, C), negatif (negative, N) ve sıfır (zero, Z). Bir n bitlik işlem sonucu sıfırsa Z biti 1 olur. Bunu sağlayan devre şekildeki sıfır algılama (zero detect) devresidir.

Soru (hocadan): Bütün bitler 0 iken çıkışı 1 olan tek bir kapı söyleyin.

Cevap: İlk öneri OR kapısıydı. Bütün girişler 0 iken OR çıkışı 0 olur, istenen ise 1'dir, bu yüzden olmaz. Doğrusu NOR kapısıdır. Bütün girişler 0 iken OR 0 verirdi, NOR 1 verir. Girişlerden biri bile 1 olursa çıkış 0 olur.

3.9 Register Kümesi, Fonksiyon Birimi ve Dış Bağlantılar

Şekilde iki blok mavi ile renklendirilmiştir. Register kümesi (register file) bloğunda işleme katılacak kaynak işlenenler seçilir. Fonksiyon biriminde ise seçilmiş register içeriği üzerinde aritmetik, lojik ya da kaydırma işlemi yapılır.

İki blok arasından dışarıya iki bağlantı alınmıştır: A yolundan address out, B yolundan data out. Bunlar, data in ile birlikte, çevre birimleriyle ve bellekle haberleşmek için kullanılan uçlardır. Address out ile bellekte bir adres seçilir. Bu adrese bir veri yazılabilir ya da oradaki veri okunup iç register'a taşınabilir.

4. Aritmetik Devre Tasarımı

Dersin ilk hedefi, fonksiyon biriminin içindeki n bitlik aritmetik lojik birimi tasarlamaktır. Bu birimin girişleri n bitlik A ve B sayılarıdır (data input A, data input B). İşlem, operation select ve mode select uçları ile elde girişi (carry in) ile seçilir. Sonuç n bitlik G çıkışında görülür. Bir de elde çıkışı (carry out) vardır.

4.1 Temel Yapı: Paralel Toplayıcı

Tasarım daha önce görülen n bitlik paralel toplayıcıya (parallel adder) dayanır. Paralel toplayıcı tam toplayıcılardan (full adder) kurulur: Dört bitlik paralel toplayıcı için dört tam toplayıcı kullanılır. Tam toplayıcı, elde biti ile birlikte üç biti toplayabilen devredir.

En düşük anlamlı tam toplayıcı A_0 ile B_0'ı ve dışarıdan gelen elde girişini toplar, S_0 toplam bitini üretir. Oluşan elde, bir yüksek anlamlı tam toplayıcının elde girişine bağlanır. O da A_1 ile B_1'i toplayıp S_1'i üretir ve kendi eldesini yandaki bite verir.

Asıl fikir şudur: Paralel toplayıcının girişlerine farklı kombinasyonlar uygulayarak devrenin yaptığı işi değiştirmek. X girişine n bitlik A sayısı doğrudan verilir. B sayısı ise önce bir lojik bloktan, **B giriş lojiğinden (B input logic)**, geçirilir. Bu bloğun örnekte iki seçme ucu vardır, ama sayısı bir, üç, daha az ya da daha çok olabilir. B doğrudan verilebilir, tümleyeni verilebilir ya da hiç gönderilmeyip 0 verilebilir. Elde girişi de toplama katıldığı için çıkış şudur:

$$G = X + Y + C_{in}$$

4.2 Fonksiyon Tablosu

Kendi tanımladığımız işlemleri yapan bir devre istiyoruz. Bunun için önce işlemleri ve her işlem için seçme uçlarının ne olacağını gösteren bir tablo hazırlanır. Bu örnekte sekiz aritmetik işlem istenmektedir. Amacımız, $X = A$ iken tablodaki her satır için Y'nin ne olması gerektiğini bulmak ve B giriş lojiğini buna göre tasarlamaktır.

S_1S_0	Y	$C_{in} = 0$	$C_{in} = 1$
00	hepsi 0	$G = A$ (aktarım)	$G = A + 1$ (artırma)
01	B	$G = A + B$ (toplama)	$G = A + B + 1$

S_1S_0	Y	$C_{in}=0$	$C_{in}=1$
10	$\overline{B}$	$G=A+\overline{B}$	$G=A+\overline{B}+1$ (çıkarma, $A-B$)
11	hepsi 1	$G=A-1$ (azaltma)	$G=A$ (aktarım)

Satırlar tek tek şöyle okunur. Hoca her satırda aynı noktayı tekrarladı: Sağ sütundaki "+1" B'den değil, C_{in}'den gelir.

- **00:** $C_{in}=0$ iken çıkışta A, $C_{in}=1$ iken $A+1$ görmek istiyoruz. İşlemlerde B hiç yok. Bu yüzden Y'ye n tane 0 (all zeros) uygulanmalıdır.
- **01:** $A+B$ ve $A+B+1$ isteniyor. Toplanan sayılar A ve B olduğu için B giriş lojiği B üretmelidir.
- **10:** $A+\overline{B}$ ve $A+\overline{B}+1$, yani $A-B$ isteniyor. B giriş lojiği $\overline{B}$ üretmelidir.
- **11:** $A-1$ ve A isteniyor. Burası en dikkat edilecek satırdır.

4.3 $A-1$ İşlemi: Neden "Hepsi 1"?

Bu fonksiyonu gerçekleştirmek için Y girişine bütün bitleri 1 olan kombinasyon (all ones) uygulanır. Hocanın sekiz bitlik örneğini adım adım yazalım:

- Verilen: 11111111.
- İşaret biti 1 olduğu için sayı negatiftir.
- Onluk karşılığını bulmak için ikiye tümleyeni alınır: bitler ters çevrilir 00000000, 1 eklenir 00000001 (ara adım eklendi).
- Büyüklük 1 olduğundan sayı -1'dir.

Yani ikiye tümleyen (two's complement) gösteriminde bütün bitlerin 1 olması -1 demektir. $A+(-1)$ işlemi $A-1$ sonucunu, yani azaltma (decrement) işlemini verir. $C_{in}=1$ olduğunda ise $-1+1=0$ olur ve çıkışta yalnız A görülür.

> **Hocanın vurgusu:** Bu tabloda en az bilinen ve en dikkat edilmesi gereken nokta budur. $C_{in}=0$ iken $A-1$ yapabilmek için B kanalından -1 sayısının gelmesi gerekir.

4.4 Doğruluk Tablosu ve Karnaugh Haritası

Y'nin alacağı değerler bellidir: hepsi 0, B, $\overline{B}$, hepsi 1. Şimdi B_i'yi de tabloya koyarak tek bir bit için B giriş lojiğinin doğruluk tablosunu yazarız. 01 satırında Y_i, B_i neyse o olur. 10 satırında tümleyeni olur. 11 satırında B_i 0 da olsa 1 de olsa çıkış 1'dir.

S_1	S_0	B_i	Y_i
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Bu değerler Karnaugh haritasına (Karnaugh map) taşınır:

S_1S_0	$B_i=0$	$B_i=1$
00	0	0
01	0	1
11	1	1
10	1	0

Haritada amaç, olabilecek en büyük grupları ve en az sayıda grubu oluşturmaktır. İki ikili grup çıkar: 01 ve 11 satırlarının $B_i = 1$ sütunu $S_0 B_i$ terimini, 11 ve 10 satırlarının $B_i = 0$ sütunu $S_1 \overline{B_i}$ terimini verir. Sadeleşmiş ifade:

$$Y_i = B_i S_0 + \overline{B_i} S_1$$

4.5 Devrenin Tamamı

Bu ifade her bit için bir lojik bloktur. Blok, her tam toplayıcının önüne konur: B_0 için olanı en düşük anlamlı tam toplayıcının önüne, B_1 için olanı bir sonrakinin önüne, böyle devam eder. C_{in} en düşük anlamlı bite elde olarak girer. Her bitin elde çıkışı bir üst bitin elde girişine bağlanır. Sonraki tam toplayıcı A_1 ile B giriş lojiğinden geçmiş B_1'i toplar ve G_1'i üretir. Hocanın dediği gibi yapı aslında kolaydır. Tasarlanması gereken tek şey, verilen işlemlere göre B giriş lojiğidir.

5. Alıştırma 1: Hem A Hem B Giriş Lojiği Olan Devre

Hoca kitabın bölüm sonu sorularından örnekler çözdü. İlk soru şudur: İki seçme değişkeni S_1, S_0 ve iki n bitlik veri girişi A, B olan, C_{in} ile birlikte aşağıdaki sekiz aritmetik işlemi üreten devreyi tasarlayın.

$S_1 S_0$	$C_{in} = 0$	$C_{in} = 1$
00	$F = A + B$	$F = A + \overline{B} + 1$ ($A - B$)
01	$F = \overline{A} + B$	$F = \overline{A} + B + 1$ ($B - A$, ters çıkarma)
10	$F = A - 1$	$F = A + 1$
11	$F = \overline{A}$	$F = \overline{A} + 1$

C_{in} burada bir fonksiyon seçme ucu gibi davranır. İki seçme ucunun yanına C_{in} eklenince üç seçme ucu varmış gibi olur ve sekiz işlem tanımlanabilir.

5.1 Önceki Tasarımdan Farkı

Soru (hocadan): Tabloya bakınca önceki tasarıma göre nasıl bir fark seziyorsunuz?

Cevap: A'nın önünde de bir lojik devreye ihtiyaç var. Çünkü tabloda $\overline{A}$ geçiyor ve bu ancak A'nın önüne de bir lojik devre koyarak elde edilir.

Böylece yapı şöyle olur: n bitlik A sayısı bir A giriş lojiğinden geçerek X girişini, n bitlik B sayısı bir B giriş lojiğinden geçerek Y girişini oluşturur. İki bloğu da aynı S_1, S_0 uçları kontrol eder.

Yapılması gereken ikinci analiz, C_{in}'in bir önceki kombinasyonu etkileyip etkilemediğidir. 00 satırına bakalım: $C_{in} = 0$ iken $A + B$, $C_{in} = 1$ iken $A + \overline{B} + 1$ isteniyor. C_{in} yalnız "+1" eklemiyor, B'nin biçimini de değiştiriyor. Demek ki B giriş lojiğinin bir kontrol ucu daha olmalıdır: C_{in}. A giriş lojiğinde ise C_{in} değişince A'nın biçimi değişmiyor, bu yüzden orada böyle bir uca gerek yoktur.

5.2 X_i ve Y_i Tablosu

Hoca seçme uçları ve C_{in} için X_i, Y_i ve F tablosunu öğrencilerle birlikte doldurdu:

$S_1 S_0$	C_{in}	X_i	Y_i	F
00	0	A_i	B_i	$A + B$
00	1	A_i	$\overline{B_i}$	$A + \overline{B} + 1$
01	0	$\overline{A_i}$	B_i	$\overline{A} + B$

S_1S_0	C_{in}	X_i	Y_i	F
01	1	$\overline{A_i}$	B_i	$\overline{A}+B+1$
10	0	A_i	1 (hepsi 1)	$A-1$
10	1	A_i	0 (hepsi 0)	$A+1$
11	0	$\overline{A_i}$	0	$\overline{A}$
11	1	$\overline{A_i}$	0	$\overline{A}+1$

10 satırında hoca, önceki bölümdeki mantığı öğrencilere yeniden sordu: $A-1$ için B kanalından -1 gelmeli, yani bütün bitler 1 olmalıdır. Sağdaki $A+1$ için ise:

Soru (hocadan): $A+1$ işlemi için Y_i ne olmalı?

Cevap: Tamamen 0 gelmeli. Buradaki 1 C_{in}'den gelir.

> **Ders sonrası not:** Hoca 11 satırının $C_{in}=1$ durumunda "A komplement ile 0'ı toplayarak A artı 1 elde edebilirim" dedi. Doğrusu $\overline{A}+1$'dir. Bu, A'nın ikiye tümleyenidir ($-A$), $A+1$ değildir. Python denemesinde bu satır 4 bitlik bütün değerler için $-A$ verdi.

5.3 A Giriş Lojiği

A giriş lojiği için C_{in} gerekmediğinden S_1, S_0, A_i ile üç değişkenli bir doğruluk tablosu yazılır. 00 ve 10 satırlarında $X_i=A_i$, 01 ve 11 satırlarında $X_i=\overline{A_i}$ olur.

S_1S_0	$A_i=0$	$A_i=1$
00	0	1
01	1	0
11	1	0
10	0	1

Bu Karnaugh haritasından iki terim çıkar. Öğrenciler sohbette bunun XOR olduğunu hemen gördü:

$$X_i=\overline{S_0}A_i+S_0\overline{A_i}=S_0\oplus A_i$$

5.4 B Giriş Lojiği

B giriş lojiğinde C_{in} de bir değişkendir, bu yüzden C_{in}, S_1, S_0, B_i ile dört değişkenli bir tablo gerekir. Hoca sadeleşmiş ifadenin biraz uzun olduğunu söyleyerek yalnız sonucu yazdı ve tablo ile haritayı öğrencilere bıraktı. Bölüm 5.2'deki tablodan elde edilen harita şudur (ara adım eklendi):

$C_{in}S_1 \setminus S_0B_i$	00	01	11	10
00	0	1	1	0
01	1	1	0	0
11	0	0	0	0
10	1	0	1	0

$$Y_i=\overline{C_{in}}\,\overline{S_1}B_i+\overline{C_{in}}S_1\overline{S_0}+\overline{S_1}S_0B_i+C_{in}\overline{S_1}\,\overline{S_0}\,\overline{B_i}$$

> **Ders sonrası not:** Altyazıda son terim "carry in, S1 değil, BI değil" diye geçiyor, yani $\overline{S_0}$ duyulmuyor (01:39:26). O biçimle $S_1S_0C_{in}=011$ satırı yanlış çıkar: İstenen $Y_i=B_i$ iken ifade 1 verir. Doğrusu yukarıdaki dört harfli son terimdir. Yukarıdaki ifade ve $X_i=S_0\oplus A_i$ ile sekiz işlemin hepsi Python'da 4 bitlik bütün A ve B değerleri için

doğru çıktı. En az dört terim gerektiği de hesapla doğrulandı. Birinci terim yerine $\overline{C_{in}}\,\overline{S_0}B_i$ yazılan çözüm de aynı uzunluktadır ve aynı fonksiyonu verir.

6. Alıştırma 2: Dört Bitlik Aritmetik Devre

İkinci soru benzer biçimdedir: İki seçme değişkenli (S_1, S_0), verilen tablodaki işlemleri üreten dört bitlik bir aritmetik devre tasarlayın. Tipik bir bit katının ve en düşük anlamlı bit katının lojik diyagramını çizin. Hoca sorunun bu çizim kısmında önemli bir nokta olduğunu söyledi. Bölüm 8 bu noktayı açıklar.

Tasarıma başlamadan iki soru sorulur:

- **A ve B giriş lojiklerinin ikisi de gerekli mi?** Tabloda üçüncü satırda A yoktur, yani A'nın 0 yapılması gerekir. Demek ki A giriş lojiğine ihtiyaç vardır.
- **C_{in}, A ve B'nin biçimini etkiliyor mu?** Hayır. Her satırda $C_{in} = 0$ ve $C_{in} = 1$ sütunlarında A ve B aynı biçimdedir. Örneğin bir satırda iki tarafta da B 0'dır, üçüncü satırda iki tarafta da A 0 ve B tümleyen biçimindedir.

Bu yüzden devre öncekinden daha basittir: C_{in} giriş lojiklerine bağlanmaz. Hoca bu soruyu çözmeyi öğrencilere bıraktı.

7. Alıştırma 3: Fonksiyondan İşlemi Bulma

Üçüncü soruda sorunun veriliş biçimi değişir. Tasarım istenmez, devre verilir. Her tam toplayıcının X_i ve Y_i girişlerinin lojiği şu Boole fonksiyonlarıyla belirtilmiştir. Burada S seçme değişkeni, C_{in} giriş eldesi, A_i ve B_i ise i. katın veri girişleridir:

$$X_i = A_i, \qquad Y_i = \overline{B_i}S + B_i\overline{C_{in}}$$

İstenen, S ve C_{in}'in dört kombinasyonu için yapılan aritmetik işlemi bulmaktır. Hoca buna tersine mühendislik dedi. Tek seçme ucu olduğu için C_{in}'i de bir değişken gibi tabloya yazdı. Çıkış yine $F = X + Y + C_{in}$'dir ve $X_i = A_i$ olduğu için yalnız Y_i değişir. Değerleri yerine koyalım:

- $S = 0$, $C_{in} = 0$**:** $Y_i = \overline{B_i} \cdot 0 + B_i \cdot 1 = B_i$. Çıkış $F = A + B$ (toplama).
- $S = 0$, $C_{in} = 1$**:** $Y_i = \overline{B_i} \cdot 0 + B_i \cdot 0 = 0$. Çıkış $F = A + 1$ (artırma). Öğrenciler önce $A + B$ dedi, sonra düzeltti. Hoca, toplamı yazarken C_{in}'in mutlaka yazılması gerektiğini vurguladı.
- $S = 1$, $C_{in} = 0$**:** $Y_i = \overline{B_i} + B_i = 1$. Y'de bütün bitler 1'dir, yani -1 gelir. Çıkış $F = A - 1$ (azaltma).
- $S = 1$, $C_{in} = 1$**:** $Y_i = \overline{B_i} + 0 = \overline{B_i}$. Çıkış $F = A + \overline{B} + 1 = A - B$ (çıkarma).

S	C_{in}	Y_i	$F = X + Y + C_{in}$
0	0	B_i	$A + B$
0	1	0	$A + 1$
1	0	1 (hepsi 1)	$A - 1$
1	1	$\overline{B_i}$	$A - B$

Ders sonrası not: Altyazıda bu satır için "B kanalından A-1 sayısı gelmiş" geçiyor (01:48:52). B kanalından gelen sayı -1'dir, $A - 1$ ise çıkıştır. Dört işlem Python'da 4 bitlik bütün A ve B değerleri için doğru çıktı.

8. Giriş Lojiğindeki C_{in} Nereden Gelir?

Hoca Alıştırma 1'in çizimini atladığını fark edip buna döndü. B giriş lojiğine bir değişken olarak bağlanan C_{in} nereden gelir? Her blok için ayrı bir tam toplayıcıdan mı alınır?

Çizim şöyledir: En düşük anlamlı bitten başlayarak her tam toplayıcının önünde bir A giriş lojiği ve bir B giriş lojiği vardır. S_0 ve S_1 her iki bloğun da kontrol ucudur. C_{in} ise yalnız B

giriş lojiğinde vardır. A_0 ve B_0'ın önündeki bloklar için durum budur. A_1 ve B_1 için de aynı S_0, S_1 ve aynı C_{in} kullanılır.

Bu C_{in}, en düşük anlamlı bite dışarıdan bağlanan elde girişidir. Bütün B giriş lojiklerinin önüne bir kontrol işareti olarak o götürülür. Tam toplayıcıların ürettiği elde çıkışları ise klasik uygulamadaki gibi bir sonraki bitin elde girişine bağlanır. Sistem, en düşük anlamlı bitten en yüksek anlamlı bite doğru zincir yapıda çalışır. Bu ara eldeler kontrol için kullanılmaz.

> **Hocanın vurgusu:** B giriş lojiğinde kullanılan C_{in}'in devrenin içinde oluşan elde bitleriyle hiçbir ilgisi yoktur. Bunları karıştırmayın.

9. Lojik Blok Tasarımı

Hoca lojik bloğun çok kolay olduğunu söyledi. İşlem seçme uçları (operation select) AND, OR, XOR ve NOT işlemlerinden birini seçer:

S_1S_0	Çıkış
00	$A_i \wedge B_i$ (AND)
01	$A_i \vee B_i$ (OR)
10	$A_i \oplus B_i$ (XOR)
11	$\overline{A_i}$ (NOT)

Bunun için 4'ten 1'e bir MUX ve ilgili lojik kapılar yeterlidir. A_i, B_i bitleri arasında iki girişli bir AND, iki girişli bir OR ve bir XOR kapısı vardır. NOT işlemi ise yalnız A girişi üzerinde yapılır.

Soru (hocadan): Sekiz bitlik iki sayı arasında işlem yapacaksam kaç MUX kullanmalıyım?

Cevap: İlk öneri dörttü, ama doğrusu sekizdir. Dörtten bire MUX tek bit çıkış verir, sekiz bit için sekiz çıkış gerekir. Her MUX'un önünde de bu kapılardan birer takım bulunur.

10. Aritmetik ve Lojik Bloğu Birleştirmek: ALU

Kitap aritmetik devrenin bir katını (one stage of arithmetic circuit) ve lojik devrenin bir katını (one stage of logic circuit) yan yana koyar. Bir kat, A'nın ve B'nin i. biti üzerinde işlem yapan bloktur. İki blok da A_i ve B_i'yi giriş olarak alır.

- **Aritmetik kat:** İki seçme ucu (S_0, S_1) ve C_{in} ile sekiz aritmetik işlem tanımlar.
- **Lojik kat:** Kendi S_0, S_1 uçları ayrı bağlanmaz, yukarıdaki S_0 ve C_{in}'den alınır.
- **2'den 1'e MUX:** İki bloğun çıkışı bu MUX'a girer. Seçme ucu S_2'dir. Hoca buna mod seçme (mode select) dedi. Aritmetik devreden mi lojik devreden mi çıkış alınacağını belirleyen yüksek anlamlı seçme ucudur.

  > **Ders sonrası not:** Hoca bağlantıyı "S0, S1'i sırasıyla yukarıdaki S0 ve carry in'den almış" diye anlattı (02:13:19). Bu kelimesi kelimesine lojik $S_0 \leftarrow S_0$, lojik $S_1 \leftarrow C_{in}$ demektir. İşlem kodlarıyla (14. bölüm) tutarlı bağlantı terstir: lojik $S_1 \leftarrow S_0$, lojik $S_0 \leftarrow C_{in}$. Python denemesinde bu bağlantı ile aritmetik tablo birlikte, FS = 0000-1011 kodlarının hepsini 4 bitlik bütün değerler için doğru verdi. Kısa not da aynı bağlantıyı yazar.

Böylece ALU devresi elde edilir. Aritmetik blok ile lojik blok paralel çalışır, aynı anda gelen A ve B için ikisi de sonuç üretir. Mod seçme ucu hangisinin çıkışa aktarılacağını belirler. $S_2 = 0$ iken sekiz aritmetik işlem, $S_2 = 1$ iken yalnız S_0 ve C_{in} ile seçilen dört lojik işlem yapılır:

S_2	S_1	S_0	C_{in}	İşlem
0	0	0	0	$G = A$
0	0	0	1	$G = A + 1$
0	0	1	0	$G = A + B$
0	0	1	1	$G = A + B + 1$

S_2	S_1	S_0	C_{in}	İşlem
0	1	0	0	$G = A + \overline{B}$
0	1	0	1	$G = A + \overline{B} + 1$
0	1	1	0	$G = A - 1$
0	1	1	1	$G = A$
1	$\times$	0	0	$G = A \wedge B$
1	$\times$	0	1	$G = A \vee B$
1	$\times$	1	0	$G = A \oplus B$
1	$\times$	1	1	$G = \overline{A}$

11. Kombinasyonel Kaydırıcı (Shift Unit)

Aritmetik ve lojik işlemler tamam, sırada kaydırma işlemleri var. Daha önce D flip-floplarla, birinin çıkışı diğerinin girişine gelecek biçimde senkron bir kaydırma register'ı kurulmuştu. Burada ise tasarım kombinasyonel devreyle yapılır. Yani hiç flip-flop kullanılmaz, devre saate bağlı çalışmaz.

> **Hocanın vurgusu:** Tasarımın kombinasyonel devre elemanlarıyla yapıldığına dikkat edin.

Yukarıda 4 bitlik bir B yolu olsun. Yolun üzerine yerleştirilecek birim için en basit çözüm MUX'tur. Dört tane 4'ten 1'e MUX kullanılır, hepsi aynı iki seçme ucuyla kontrol edilir. Her MUX'un dört girişinden yalnız üçü kullanılır:

S	İşlem	H_3	H_2	H_1	H_0
00	Değişiklik yok (no change)	B_3	B_2	B_1	B_0
01	Sağa kaydırma	seri giriş	B_3	B_2	B_1
10	Sola kaydırma	B_2	B_1	B_0	seri giriş

Hoca tabloyu MUX'ların girişlerinden okuyarak kurdu. 00 iken çıkışta girişteki neyse o görülür. 01 iken 1 numaralı girişlere bakılır: H_2 girişini B_3'ten, H_1 girişini B_2'den alır. B_3'ü H_2'ye göndermek, sağa bir bit kaydırmak demektir. 10 iken 2 numaralı girişlere bakılır: H_3 girişini B_2'den, H_2 girişini B_1'den alır. Bu da sola kaydırmadır. Tablonun altyazıda sayılmayan hücreleri aynı kuralla doldurulmuştur (ara adım eklendi). Python denemesinde 01 sağa, 10 sola bir bit kaydırma verdi.

Soru: Bu yapı yalnız bir bit mi kaydırabiliyor? Bir seferde üç bit kaydırmasını isteyemez miyiz?

Cevap: Bu yapıda olmaz, bu tasarım yalnız bir bit kaydırır. Bloğu arka arkaya birkaç kez çalıştırarak yapılabilir. Bir sonraki örnek olan barrel shifter ise başka bir çözümdür.

12. Barrel Shifter

Dört bitlik barrel shifter'ın iki seçme ucu vardır. 00 döndürme yok (no rotation) demektir, Y_3'te D_3, Y_2'de D_2 görülür. Diğer kombinasyonlar sırasıyla sola 1, 2 ve 3 konum döndürmedir (rotate left):

S	İşlem	Y_3	Y_2	Y_1	Y_0
00	Döndürme yok	D_3	D_2	D_1	D_0
01	Sola 1 konum	D_2	D_1	D_0	D_3
10	Sola 2 konum	D_1	D_0	D_3	D_2
11	Sola 3 konum	D_0	D_3	D_2	D_1

Bir konum döndürmede Y_3'e D_2 gelir. Hoca Y_0'a dikkat çekti: Döndürme olduğu için en düşük anlamlı çıkışa D_3 gelir. 4 konum döndürmek zaten "döndürme yok" ile aynı olacağı için dört bit için bu tablo yeterlidir. Hocaya göre bu yapının barrel shifter diye geçmesinin nedeni, az

önceki soruda sorulan gibi birden fazla kaydırmayı yapabilmesidir. 2 ve 3 konumluk satırların hücreleri aynı kuralla yazılmıştır (ara adım eklendi) ve Python ile doğrulandı.

Tablo çıkışa hangi girişin bağlanacağını gösterdiği için devreye geçmek kolaydır: Her çıkış için bir 4'ten 1'e MUX kullanılır (dört girişi, iki seçme ucu, bir çıkışı vardır) ve girişleri tablonun o sütunundaki sıraya göre bağlanır. İşlem döndürme olduğu için bu yapının seri giriş kanalı yoktur.

13. Genel Datapath ve 16 Bitlik Kontrol Kelimesi

Dersin başındaki yapıya dönelim. Bir register kümesi ve bir fonksiyon birimi vardı, bunların bileşimine datapath birimi dedik. Haftanın amacı bu birimi oluşturmaktı. Kitap şimdi aynı yapıyı çok genel bir biçimde gösteriyor.

13.1 $2^m \times n$ Gösterimi

Register kümesinin üstünde $2^m \times n$ yazar. Örneğin içinde 8 register varsa ve her biri 16 bitlikse $2^3 \times 16$ yazılır. Burada m, register seçme uçlarının sayısıdır: 8 register için 3 adres ucu gerekir. Bu uçlar, hangi register'ın işleme katılacağını seçen MUX'ların seçme uçlarıdır.

Yapı önceki şekille aynı mantıktadır:

- A adresi ve B adresi seçilince seçilen register'ın içeriği n bitlik A data ve B data yollarına aktarılır.
- B yolunda 2'den 1'e bir MUX (tek bitlik MB select) veriyi register'dan mı sabit sayı girişinden mi alacağını seçer. Çıkışı fonksiyon birimine gider.
- Fonksiyon birimi, tasarladığımız ALU ve kaydırma biriminden oluşur. Seçilen işlemi yapar, n bitlik F çıkışını üretir ve F'ye göre durum bitlerini oluşturur: taşma var mı, elde var mı, sonuç sıfır mı, sonuç negatif mi.
- MUX D (2'den 1'e, tek bitlik seçme ucu) fonksiyon birimi ile data in arasında seçer. Çıkışı yukarı dönerek register kümesinin D data (destination data) girişine bağlanır.
- D data bütün register'ların girişine gider. Sonucun hangisine yazılacağını m bitlik hedef adresi (D address) seçer.

13.2 Sekiz Register'lı Datapath İçin Kontrol Kelimesi

Yapının özelleşmiş biçiminde register kümesinde 8 tane n bitlik register vardır. O hâlde hedef, A ve B adresleri için 3'er bit gerekir. Bütün kontrol bitleri arka arkaya dizilince bir kontrol kelimesi (control word) elde edilir:

Bitler	Alan	Boyut	Görev
15-13	DA	3	Hedef register adresi
12-10	AA	3	A kaynak register adresi
9-7	BA	3	B kaynak register adresi
6	MB	1	MUX B'nin seçme ucu
5-2	FS	4	Fonksiyon seçimi
1	MD	1	MUX D'nin seçme ucu
0	RW	1	Register'a yazma (en düşük anlamlı bit)

Toplam $3+3+3+1+4+1+1=16$ bittir.

> **Ders sonrası not:** Hoca MB için önce "7, 6" dedi (02:32:28). 7 numaralı bit BA'ya aittir. MB yalnız 6 numaralı bittir. Hoca da sonra "6 numaralı bit" dedi.

Bu kelime, işlenenleri seçmek, fonksiyonu seçmek ve sonucun nereye yazılacağını seçmek için gereken bütün bitleri içerir. Bu tasarıma özgüdür: 8 register'lı bir datapath birimi için tanımlanmıştır. Artık yapmak istediğimiz bütün işlemleri bu kelime üzerinden ifade edebiliriz.

14. Kontrol Kelimesinin Kodlama Tablosu

Kitap, kontrol kelimesinin kodlamasını (encoding), yani hangi kodun ne iş yaptığını gösteren bir tablo verir.

> **Hocanın vurgusu:** Bu tabloları ezbere bilmek gerekmez. Tablolar ve devre şeması önümüzde olur, önemli olan tabloyu okuyabilmektir.

- **DA, AA, BA:** Register kümesindeki register'ları seçer. 000 R0'ı, 100 R4'ü seçer. R0 = 000'dan R7 = 111'e kadar gider.
- **MB:** 0 ise veri register kümesinden, 1 ise sabit sayı girişinden gelir.
- **MD:** 0 ise fonksiyon biriminden, 1 ise dışarıdan, data in kanalından gelir.
- **RW:** 0 ise yazma yoktur (no write), 1 ise sonuç register'a yazılır. Her işlem sonucu register'a yazılmak zorunda değildir.

FS kodları, tasarım sırasında oluşturulan tablodan alınmıştır. İlk 8 kod aritmetik, sonraki 4 kod lojik, altındaki 3 kod kaydırma işlemidir:

FS	İşlem	FS	İşlem
0000	$F = A$	1000	$F = A \wedge B$
0001	$F = A + 1$	1001	$F = A \vee B$
0010	$F = A + B$	1010	$F = A \oplus B$
0011	$F = A + B + 1$	1011	$F = \overline{A}$
0100	$F = A + \overline{B}$	1100	$F = B$
0101	$F = A + \overline{B} + 1$	1101	$F = \text{sr } B$
0110	$F = A - 1$	1110	$F = \text{sl } B$
0111	$F = A$		

> **Ek bilgi (derste anlatılmadı):** Tablolar karşılaştırılınca 4 bitlik FS'nin büyük olasılıkla çözülerek G select, H select ve MF select uçlarına dağıtıldığı görülür. FS'nin en üst iki biti 11 olan satırlar shifter işlemleridir ve en alt iki bitleri 00, 01, 10 ile Bölüm 11'deki değişiklik yok, sağa, sola kodlarıyla aynıdır. Geri kalan satırlarda dört bit, Bölüm 10'daki $S_2 S_1 S_0 C_{in}$ sırasına uyar.

15. Mikro İşlem Örnekleri

Hoca kitabın örnek tablosunu birkaç satır üzerinden açıkladı. Bir mikro işlem (micro-operation) tanımlanır ve 16 bitlik kontrol kelimesinin alanlarının ne olacağı belirlenir. Register adlarının yazıldığı bu biçime sembolik gösterim (symbolic notation) denir.

> **Ek bilgi (derste anlatılmadı):** Mikro işlem, büyük olasılıkla tek bir saat çevriminde register içerikleri üzerinde yapılan temel işlem anlamında kullanılmaktadır. Bir kontrol kelimesi de böyle tek bir işlemi tarif eder.

Mikro işlem	DA	AA	BA	MB	FS	MD	RW
$R1 \leftarrow R2 - R3$	R1	R2	R3	Register	$\overline{A} + \overline{B} + 1$	Function	Write
$R4 \leftarrow \text{sl } R6$	R4	—	R6	Register	sl B	Function	Write
$R7 \leftarrow R7 + 1$	R7	R7	—	—	$A + 1$	Function	Write
$R1 \leftarrow R0 + 2$	R1	R0	—	Constant	$A + B$	Function	Write
Data out $\leftarrow R3$	—	—	R3	Register	—	—	No Write
$R4 \leftarrow$ Data in	R4	—	—	—	—	Data in	Write

Mikro işlem	DA	AA	BA	MB	FS	MD	RW
$R5 \leftarrow 0$	R5	R0	R0	Register	$A \oplus B$	Function	Write

15.1 Çıkarma: $R1 \leftarrow R2 - R3$

R2'den R3'ü çıkarıp sonucu R1'e yazmak istiyoruz. Alanlar sırayla doldurulur:

- Hedef R1 olduğu için DA = R1.
- A yoluna R2, B yoluna R3 konur.
- MUX B sabit sayıyı değil, register'dan gelen değeri kullanır.
- FS'de çıkarmaya karşılık gelen $A + \overline{B} + 1$ seçilir. İkili kod istenirse tablodan bakılır: 0101.
- İşlem fonksiyon biriminde yapıldığı için MUX D fonksiyon birimini seçer.
- Sonuç R1'e yazılacağı için yazma etkinleştirilir (RW = 1).

15.2 Kaydırma: $R4 \leftarrow$ **sl** $R6$

Tabloya bakılırsa sola kaydırma yalnız B işleneni üzerinde tanımlıdır. Bu, işlemin B yolu üzerinde yapıldığı anlamına gelir. Bu yüzden hedef R4 olurken kaynak R6, B adresi üzerinden seçilir. Veri register'dan gelir. FS'de sl B seçilir, kodu 1110'dur. Çıkış fonksiyon biriminden gelir ve R4'e yazılacağı için yazma etkinleştirilir.

Soru: Bu örnekte kontrol kelimesini yazarken A adresine bir şey yazmanın sakıncası yok değil mi?

Cevap: Hiç önemi yok. Orası don't care'dir ve tire ile geçilir. Mutlaka bir şey yazılması gerekiyorsa 000 yazılması tercih edilir.

15.3 Sabit Sayı: $R1 \leftarrow R0 + 2$

R1'e R0 + 2 yazılacak. Burada komutun içinde verilmiş bir işlenen (immediate operand) vardır. Böyle bir değer ancak sabit sayı girişinden alınabilir. Bu yüzden MB, Constant olarak işaretlenir.

15.4 Dışarıya Veri: Data out $\leftarrow R3$

R3'ün içeriği data out ile dışarıya, belki belleğe ya da bir porta yazılacak. Şekle bakınca data out B yolu üzerindedir. A yolu üzerinden adres gönderilir, ama burada adres bilgisi istenmemiştir. Register'a kaydetme olmadığı için hedef adresi yoktur. A yolu kullanılmaz. Yalnız B yolu üzerinden R3 seçilir.

İşlem data out'un alındığı yerde sonlanır: Aşağıdaki blokların hiçbiri, ne fonksiyon birimi ne MUX D kullanılır. Bu yüzden FS ve MD tire ile geçilir, RW ise no write olarak işaretlenir.

15.5 Dışarıdan Veri: $R4 \leftarrow$ Data in

Data in'den gelen veri R4'e yazılacak. Hoca burada devre şemasının akılda tutulmasını istedi. Veri doğrudan MUX D'ye gelir. Bu yüzden önemli olan üç alan vardır: MD data in'i seçmeli, RW yazmayı etkinleştirmeli ve DA R4'ü göstermelidir. A adresi, B adresi, MUX B ve fonksiyon kullanılmayan bloklardır. Tire ile geçilirler, kod yazılması gerekiyorsa 0 yazılır.

15.6 Sıfırlama: $R5 \leftarrow 0$

Bir register'ı sıfırlamak farklı biçimlerde yapılabilir. Hocanın en uygun bulduğu yol XOR işlemidir: Bir register kendisiyle XOR'lanıp sonuç R5'e yazılırsa R5 sıfırlanır. Tablodaki satırda AA ve BA'nın ikisi de R0'dır. Kendisiyle XOR'lanan değerin her biti 0 verdiği için sonuç sıfırdır (ara adım eklendi).

Soru: Saat çevrimi bakımından XOR'un daha kısa olduğu önceki dönemde söylenmişti, burada da öyle mi?

Cevap: Bizim tasarımımızda lojik bloğun bütün işlemleri aynıdır. Ama aritmetik işlemle yapmaya göre lojik işlemle yapmak kesinlikle daha kısadır.

16. İkili Kodlu Gösterim ve Alıştırma 4

Aynı tablo ikili kodlar yazılarak da oluşturulabilir. Hocanın okuduğu ilk satırda hedef R1, A yolunda R2, B yolunda R3 seçilir. Çıkarma kodu 0101'dir, MD 0, RW 1'dir. Yani sembolik gösterim ile ikili kod aynı kelimenin iki yazılışıdır ve ikisinin de yazılabilmesi gerekir.

$$R1 \leftarrow R2 - R3: \quad 001\ 010\ 011\ 0\ 0101\ 0\ 1 = 0010100110010101$$

Bütün satırların ikili hâli, alan kodlarından şöyle çıkar (ara adım eklendi). Tire, don't care demektir:

Mikro işlem	DA	AA	BA	MB	FS	MD	RW
$R1 \leftarrow R2 - R3$	001	010	011	0	0101	0	1
$R4 \leftarrow$ sl $R6$	100	—	110	0	1110	0	1
$R7 \leftarrow R7 + 1$	111	111	—	—	0001	0	1
$R1 \leftarrow R0 + 2$	001	000	—	1	0010	0	1
Data out $\leftarrow R3$	—	—	011	0	—	—	0
$R4 \leftarrow$ Data in	100	—	—	—	—	1	1
$R5 \leftarrow 0$	101	000	000	0	1010	0	1

Bir değer yazmak gerektiğinde tire yerine 0 yazılır. Örneğin $R4 \leftarrow$ sl $R6$ kelimesi AA = 000 ile $100\ 000\ 110\ 0\ 1110\ 0\ 1$ olur.

Alıştırma 4: 11. şekildeki datapath'e, yani az önce görülen 8 register'lı devreye, verilen mikro işlemleri uygulamak için gereken 16 bitlik kontrol kelimelerini yazın. Listede register sıfırlama ve kaydırma işlemleri vardır, ayrıca bir sağa kaydırma bulunur. Hocanın hatırlatması: Kaydırma işlemlerinde kaynağın B yolu üzerinden seçildiğini unutmayın.

Dersin Toparlaması

Bu derste bir CPU'nun iki temel bloğundan datapath birimi kuruldu. Bir register kümesinden iki işlenen seçildi, B yoluna sabit sayı seçeneği eklendi. Fonksiyon biriminde ALU ve kaydırıcı paralel olarak sonuç üretti, MUX F ve MUX D ile hangi değerin register'lara döneceği belirlendi, hedef register da çözücü ile seçildi. ALU'nun içi paralel toplayıcı üzerine kuruldu: Toplayıcının Y girişine C_{in} ve seçme uçlarıyla farklı değerler vererek sekiz aritmetik işlem elde edildi, bunlara bir MUX ile lojik işlemler eklendi. Kaydırıcı ise flip-flopsuz, MUX'larla kuruldu.

Bütün bu seçme uçları 16 bitlik kontrol kelimesinde toplandı ve her mikro işlemin bu kelimeyle, sembolik ya da ikili olarak yazılabildiği görüldü. Konu burada bitmiyor: Datapath tasarlandı, ama CPU'nun içindeki kontrol devresi ve iki bloğun etkileşimi henüz görülmedi. Bir sonraki derste kontrol devresinin datapath birimiyle nasıl birlikte çalıştığı anlatılacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.